실전 1 보고서 2부 ⑧ Holdout 개선 내역

> - **과제**: 금융사 사내 문서에서 개인정보 7종을 찾는 탐지기를 규칙 기반·토큰 분류·LLM 세 방식으로 비교하고, 운영에 쓸 탐지기를 근거 있게 고른다.
> - **데이터**: 제공 `pii_synth_finance` dev 100건(정답 548개) · test 200건(정답 1,165개) / 추가 시험 데이터: 엣지 케이스 64건(정답 116개), 일상 문서 1,983건(정답 3,375개) / **멘토 holdout 187건(정답 912개)**
> - **채점**: 멘토 제공 `run_compare.py` (strict·partial 정밀도/재현율/F1, mask_rec, 라벨별·어려운 사례 재현율, 일관성)
> - **작성**: 2026-10-26 · 저장소 `hosung` 브랜치 `실전프로젝트1/` · 작업 기록 전체는 `worklog.md` (24장)

보고서 구성: 0 요약 및 결론 · **1부** ① 데이터 관찰 리포트 · ② 방식별 비교표 · ③ 오류 유형 분석 · ④ 탐지기 선정 근거 · **2부** ⑤ Holdout 데이터 관찰 · ⑥ Holdout 성능 · ⑦ Holdout 오류 분석 · ⑧ Holdout 개선 내역 · 부록 개선 과정과 데이터 사용 기록 (`report/` 폴더)

> **주의**: 이 장의 개선 후 holdout 점수는 holdout 오류를 보고 고친 결과라 **일반화 성능이 아니다.** 일반화 확인은 8-4의 dev·test·일상 문서 점수로 본다. 순서: 개선 전 채점·기록(⑥) → 오류 분석(⑦) → 개선(이 장) → holdout 재채점 → dev·test·엣지·일상 문서 재확인

8-1. 무엇을 바꿨나 (탐지기 v7 → v8)

대상	변경 (파일)	해결한 holdout 오류
1. 규칙 오탐	줄 맨 앞 `이름:` 규칙은 **문서 안에서 두 번 이상 말한 사람**이거나 서식 항목 접미사(`지·자·인·역·처·명·일·호`)로 끝나지 않을 때만 이름으로 봄. 명사 덩어리 규칙도 바로 뒤에 콜론이 오고 항목 접미사로 끝나면 제외 (`detectors/rule_based.py`)	`소명자:`, `위임인:`, `송금인:`, `배송지:` 오탐 34건
2. 규칙 오탐	직함 `심사역·조사역·검사역·감사역·상담역` 추가, 부고 단어(`장인상` 등)는 이름 아님	`심사역` 3건, `장인상` 2건
3. 규칙 경계	사전에 없는 3글자 덩어리가 "이름 + 주격 조사 `이`"로 분석되면 조사를 뗌	`안윤이`, `정별이`
4. 규칙 미탐	필드 추가(`수취인·송금인·위임인·소명자·피해자·검토자·신고인·청구인·확인자·승인자·심사역`), 사람만 올 수 있는 필드는 일반 단어와 같은 이름도 받음, **띄어 쓴 외자**(`전 찬`)	`신설`, `전 찬`
5. 규칙 형식	계좌 마지막 묶음 1자리(`056-75-340175-9`), 읍·면 바로 뒤 번지(`배방읍 252-4`), 주소 앞 우편번호(`(05299)` …)	라벨 오류 3건, 경계 오류 5건
6. 토큰 후처리	부고 단어·회사 이름 끝말(`보안`, `시스템즈` …)로 끝나는 이름 제거, 단어 중간을 자른 이름 제거(앞이 호칭·직급이면 예외: `회원박희정님`), `oo` 번호(사건·접수·주문 등) 뒤· 내선 뒤 번호와 연도-일련번호 제거, 이름 뒤 `드림`·`주격 이` 제거, 주소 앞 우편번호 포함 (`postprocess.py`)	`미르보안`, `조모상`, `미수(신)`, 사건번호 5건, 내선 2건, `시우 드림`, `오가은이`

모두 holdout 문장을 그대로 외우는 대신 **형식·문맥 규칙**으로 고쳤다(예: `소명자`라는 단어를 막지 않고 "콜론 앞 + 항목 접미사"를 막음). 한 글자 이름 `린`, 형용사 `정다운`은 고치면 일반 글자·단어를 이름으로 잡을 위험이 커서 남겼다.

고치는 중에 생긴 회귀와 수정 (구성 요소별 비교로 발견)

회귀	원인	수정
메신저에서 한 번만 말한 사람 `신승현:`을 놓침	콜론 앞 단어를 모두 서식 항목으로 봄	항목 접미사로 끝날 때만 거름
`고객: 김 과장의 김을 잡음`	1글자 첫 단어를 필드 값으로 허용	1글자는 띄어 쓴 외자일 때만
토큰 단독 test mask_rec 0.989 → 0.939	번호 문맥 정규식이 `주민등록번호의 등록번호`, `민원인의 민원`에 걸림	`oo 번호` 꼴만 보도록 수정

회귀	원인	수정
회원박희정님 (띄어쓰기 생략)을 지움	단어 중간 규칙	앞이 호칭·직급이면 예외

8-2. Holdout 개선 전 / 후 (규칙 + 토큰)

	strict P	strict R	strict F1	partial R	mask_rec	ms/문서	미탐	오탐	경계	라벨
개선 전 (v7)	0.912	0.978	0.944	0.993	0.993	48.1	3	65	14	3
개선 후 (v8)	**0.991**	**0.999**	**0.995**	**0.999**	**1.000**	54~102	**1**	**8**	0	0

구성 요소

방식	strict F1 (전 → 후)	mask_rec (전 → 후)
규칙 단독	0.923 → 0.965	0.961 → 0.984
토큰 단독	0.940 → 0.975	0.979 → 0.981
규칙 + 토큰	0.944 → **0.995**	0.993 → **1.000**

라벨별 재현율 (정밀도), 규칙 + 토큰

라벨	개선 전	개선 후
person	0.995 (0.90)	0.998 (0.99)
phone_number	1.000 (0.94)	1.000 (1.00)
email	1.000 (1.00)	1.000 (1.00)
address	0.962 (1.00)	1.000 (1.00)
주민등록번호	1.000 (0.98)	1.000 (1.00)
account_number	0.976 (0.95)	1.000 (1.00)
card_number	1.000 (1.00)	1.000 (1.00)

어려운 사례 (전 → 후): 외자 이름 0.947 → **1.000** · 계좌 형식 0.957 → **1.000** · 주소 형식 0.951 → **1.000** · 이름만 0.985 → 0.993 · 띄어 쓴 이름 0.995 → 1.000

- 남은 미탐 1: 한 글자 이름 `린` / 남은 오탐 8: 형용사 `정다운` (공지 문서에 반복) → 한계로 남김

8-3. 처리 시간 (ms/문서)

- 이번에 추가한 후처리 자체는 **문서당 0.06ms**(holdout 187건 측정) → 속도 영향 없음.
- 같은 코드도 측정할 때마다 PC 부하로 토큰 모델이 문서당 45~100ms, 규칙이 1~7ms로 흔들렸다. 개선 전 48.1ms, 개선 후 연속 측정 54~102ms → **개선 전후 차이는 측정 오차 범위**로 판단했다(바뀐 코드는 정규식 후처리뿐).

8-4. dev·test·일상 문서 재확인 — holdout에 맞춘 수정이 기존 성능을 떨어뜨렸나?

데이터	strict P (전 → 후)	strict F1 (전 → 후)	mask_rec (전 → 후)	ms/문서 (v8)
dev (100건)	0.987 → 0.987	0.994 → 0.994	1.000 → 1.000	114.8
test (200건)	0.968 → **0.976**	0.982 → **0.986**	1.000 → 1.000	61.7
엣지 케이스 (64건)	0.959 → **0.967**	0.979 → **0.983**	1.000 → 1.000	37.5
일상 문서 개선용 (989건)	0.934 → **0.952**	0.959 → **0.969**	0.987 → 0.987	33.1
일상 문서 평가용 (994건)	0.909 → **0.934**	0.942 → **0.956**	0.988 → 0.988	32.9

- **모든 데이터에서 떨어진 지표가 없다.** 정답과 정확히 일치하던 개체를 새로 놓친 경우 0건(전후 예측 비교로 확인).
- 특히 **오류를 한 번도 보지 않은 일상 문서 평가용에서도 정밀도가 0.909 → 0.934로 올랐다.** 서식 항목·회사·사건번호 오탐 제거가 holdout에만 맞춘 것이 아니라는 근거다.
- 다만 이것도 holdout 1개 세트로 고친 결과라, 진짜 일반화 성능은 **새 데이터(다음 holdout이나 실제 문서)로 다시 재야** 알 수 있다. 1부의 최종 선정(규칙 + 토큰)은 개선 전 holdout 점수(⑥)만으로도 유지된다.

8-5. 이후 추가 개선(v9) 뒤 다시 확인

holdout 개선(v8) 뒤, 새로 만든 어려운 표기 세트의 **개선용 반**(synth_hard_dev) 오류만 보고 탐지기를 한 번 더 고쳤다(v9, 부록 A-4). holdout은 이번에도 보지 않았고, 고친 뒤 점수만 다시 확인했다.

데이터	strict P (v8 → v9)	strict F1 (v8 → v9)	mask_rec (v8 → v9)
멘토 holdout	0.991 → 0.991	0.995 → 0.995	1.000 → 1.000
dev	0.987 → 0.987	0.994 → 0.994	1.000 → 1.000
test	0.976 → 0.976	0.986 → 0.986	1.000 → 1.000

- 멘토 holdout·dev·test의 예측이 v8과 같다. 정확히 맞히던 개체를 새로 놓친 경우 0건. 이어서 v10(`이`로 끝나는 이름 처리, 부록 A-3)도 세 데이터 결과가 그대로
- v9 개선 효과는 처음 보는 생성 세트에서 확인했다: 어려운 표기 평가용 반 strict F1 0.781 → 0.861, 2차 금융 업무 문서 0.855 → 0.920 (부록 A-4)

8-6. 파일

파일	내용
`detectors/rule_based.py`, `postprocess.py`, `make_union.py`	v8 (8-1의 1~6), 이후 v9 (부록 A-4)
`preds/holdout_before/`, `results/holdout_before/`	개선 전 holdout 예측·채점
`preds/holdout_after/`, `results/holdout_after/`	개선 후 holdout 예측·채점
`results/final_*`	최신(v9) 채점, holdout은 `results/final_holdout/`
`results/version_history/v8·v9·v10/`	버전별 세트 점수(summary) — 8-4·8-5 비교 근거